

StudyBuild – Project 01

Data Cleaning گزارش پروژه اول

بررسی، پاک‌سازی و آماده‌سازی دیتاست مشتریان برای تحلیل

Data Cleaning in an E-commerce Dataset

Project Report

Prepared by Sahar Baratian

1. مقدمه

داده خام معمولاً قبل از اینکه بتوان از آن برای تحلیل و تصمیم‌گیری استفاده کرد، نیاز به بررسی و پاک‌سازی دارد. وجود مقادیر خالی، رکوردهای تکراری، نوع داده نامناسب، مقادیر غیرمنطقی، Outlier و ناسازگاری بین ستون‌ها می‌تواند باعث شود نتایج تحلیل اشتباه یا گمراه‌کننده باشند.

هدف این پروژه بررسی کیفیت یک دیتاست مربوط به اطلاعات مشتریان و خریده‌های آن‌ها و تبدیل آن به یک دیتاست تمیز، استاندارد و قابل استفاده برای تحلیل‌های بعدی است.

در این پروژه ابتدا ساختار دیتاست بررسی شد و سپس مشکلات مربوط به کیفیت داده شناسایی شدند. برای هر مشکل، قبل از اعمال تغییر، دلیل تصمیم‌گیری بررسی شد تا از حذف یا تغییر غیرضروری داده‌ها جلوگیری شود.

فرایند پاک‌سازی با استفاده از زبان Python و کتابخانه Pandas در محیط Jupyter Notebook انجام شده است.

2. هدف پروژه

هدف اصلی این پروژه فقط حذف داده‌های تکراری یا جایگزین کردن مقادیر خالی نیست، بلکه هدف این است که یک دیتاست خام به یک دیتاست استاندارد و قابل اعتماد برای تحلیل‌های بعدی تبدیل شود.

در این پروژه تلاش شده است:

- مشکلات کیفیت داده شناسایی شوند.
- برای هر مشکل یک تصمیم منطقی گرفته شود.
- تغییرات اعمال شده مستند شوند.
- از حذف غیرضروری داده‌ها جلوگیری شود.
- سازگاری بین ستون‌های مختلف بررسی شود.
- در پایان کیفیت دیتاست پاک‌سازی شده دوباره بررسی شود.

این دیتاست می‌تواند در مراحل بعدی برای تحلیل اکتشافی داده‌ها (EDA)، Visualization، تحلیل رفتار مشتریان، تحلیل فروش و طراحی داشبورد مورد استفاده قرار گیرد.

3. ابزارها و کتابخانه‌های مورد استفاده

برای انجام این پروژه از Python در محیط Jupyter Notebook استفاده شد.

مهم‌ترین کتابخانه مورد استفاده، Pandas بود که برای خواندن، بررسی، پاک‌سازی و تغییر ساختار دیتاست استفاده شد.

در ابتدای Notebook کتابخانه‌های مورد نیاز Import شدند:

```
import pandas as pd
```

```
import numpy as np
```

Pandas برای عملیات اصلی Data Cleaning و NumPy برای برخی محاسبات عددی و مقایسه‌های عددی استفاده شد.

4. شناخت اولیه دیتاست

دیتاست مورد بررسی شامل اطلاعات مربوط به مشتریان، مشخصات آن‌ها، وضعیت عضویت، رفتار خرید، میزان هزینه‌کرد، روش پرداخت، دستگاه مورد استفاده و میزان رضایت مشتریان است.

هر ردیف از دیتاست نماینده یک مشتری است و هر مشتری با استفاده از ستون `customer_id` شناسایی می‌شود.

دیتاست شامل 17 ستون است:

توضیح	ستون
شناسه یکتای مشتری	customer_id
نام مشتری	first_name
جنسیت	gender
سن مشتری	age
شهر	city
استان	province
تاریخ ثبت‌نام مشتری	signup_date
سطح عضویت مشتری	membership_tier
تعداد خریدهای مشتری	purchase_count
میانگین ارزش سفارش	avg_order_value
مجموع هزینه‌کرد مشتری	total_spending
تعداد روزهای گذشته از آخرین خرید	last_purchase_days
روش پرداخت	payment_method

توضیح	ستون
دستگاه مورد استفاده	device
استفاده از تخفیف	discount_used
تعداد اقلام برگشت داده شده	returned_items
امتیاز رضایت مشتری	satisfaction_score

5. بررسی اولیه ساختار داده

در اولین مرحله، ساختار کلی دیتاست بررسی شد.

برای مشاهده چند رکورد ابتدایی از دستور زیر استفاده شد:

```
df.head()
```

برای بررسی تعداد ردیف‌ها و ستون‌ها نیز از:

```
df.shape
```

استفاده شد.

همچنین برای بررسی نام ستون‌ها:

```
df.columns
```

و برای بررسی نوع داده هر ستون:

```
df.dtypes
```

استفاده شد.

این بررسی اولیه نشان داد که دیتاست شامل اطلاعات ترکیبی از نوع عددی، متنی، دسته‌ای و تاریخی است.

در این مرحله مشخص شد که ستون `signup_date` به صورت `String` ذخیره شده و ستون `age` نیز از نوع `float64` است.

6. بررسی DUPLICATE RECORDS

یکی از اولین مواردی که بررسی شد، وجود رکوردهای تکراری بود.

برای شناسایی رکوردهای تکراری از کد زیر استفاده شد:

```
df.duplicated().sum()
```

برای مشاهده خود رکوردهای تکراری نیز می توان از:

```
df[df.duplicated()]
```

استفاده کرد.

در بررسی انجام شده، یک رکورد تکراری شناسایی شد.

از آنجا که این رکورد اطلاعات تکراری داشت و وجود آن می توانست باعث شود تعداد مشتریان یا برخی محاسبات بیشتر از مقدار واقعی نمایش داده شوند، رکورد تکراری حذف شد.

کد حذف: Duplicate

```
df = df.drop_duplicates()
```

پس از حذف، مجدداً بررسی انجام شد:

```
df.duplicated().sum()
```

نتیجه:

0

بنابراین هیچ رکورد تکراری در دیتاست باقی نماند.

7. بررسی MISSING VALUES

در مرحله بعد Missing Values در تمام ستون ها بررسی شدند.

برای این کار از دستور زیر استفاده شد:

```
df.isnull().sum()
```

نتیجه بررسی نشان داد که دو مقدار Missing در دیتاست وجود دارد:

- یک مقدار Missing در ستون age
- یک مقدار Missing در ستون total_spending

8. MISSING VALUE در ستون AGE

مقدار age برای مشتری با customer_id = 1020 خالی بود.

برای مشاهده رکورد مربوطه از کدی مشابه زیر استفاده شد:

```
df[df['age'].isnull()]
```

اطلاعات این رکورد نشان داد که مقدار واقعی سن از سایر ستون های موجود قابل استنتاج نیست.

برای بررسی توزیع سن و محاسبه Median از دستور زیر استفاده شد:

```
df['age'].describe()
```

همچنین Median سن با:

```
df['age'].median()
```

محاسبه شد.

Median برابر با 45 بود.

از آنجا که مقدار واقعی قابل استنتاج نبود، مقدار Missing با Median جایگزین شد:

```
df['age'] = df['age'].fillna(df['age'].median())
```

بنابراین:

Customer 1020: NaN → 45 برای Age

دلیل استفاده از Median این بود که Median نسبت به مقادیر بسیار بزرگ و Outlier ها مقاومت بیشتری دارد.

9. MISSING VALUE در ستون TOTAL_SPENDING

مقدار total_spending برای مشتری با customer_id = 1040 خالی بود.

ابتدا رکورد مربوط به این مشتری بررسی شد:

```
df[df['total_spending'].isnull()]
```

در این مورد مقدار واقعی total_spending را می‌توان از دو ستون دیگر محاسبه کرد.

رابطه مورد استفاده:

$$\text{total_spending} = \text{purchase_count} \times \text{avg_order_value}$$

برای: Customer 1040:

$$\text{purchase_count} = 34$$
$$\text{avg_order_value} = 37.66$$

بنابراین:

$$34 \times 37.66 = 1280.44$$

برای محاسبه این مقدار در Python می‌توان از:

```
df.loc[df['customer_id'] == 1040, 'total_spending'] = (
```

```
df.loc[df['customer_id'] == 1040, 'purchase_count']
```

```
* df.loc[df['customer_id'] == 1040, 'avg_order_value']  
)
```

استفاده کرد.

در نتیجه:

Customer 1040 = 1280.44 برای total_spending

10. کنترل MISSING VALUES پس از اصلاح

پس از اصلاح دو مقدار Missing Values، دوباره وضعیت Missing Values بررسی شد:

```
df.isnull().sum()
```

نتیجه نشان داد که هیچ Missing Value در دیتاست باقی نمانده است.

بنابراین:

Total Missing Values = 0

11. بررسی DATA TYPES

برای بررسی نوع داده ستون‌ها از:

```
df.dtypes
```

استفاده شد.

در این بررسی مشخص شد که:

```
signup_date str
```

است، در حالی که این ستون ماهیت تاریخی دارد.

همچنین age که به دلیل وجود مقدار Missing از نوع Float بود، پس از تکمیل Missing Value به Integer تبدیل شد.

12. تبدیل SIGNUP_DATE به DATETIME

ابتدا مقادیر ستون signup_date بررسی شدند:

```
df['signup_date'].head()
```

همه تاریخ‌ها از فرمت زیر پیروی می‌کردند:

YYYY-MM-DD

بنابراین ستون به Datetime تبدیل شد:

```
df['signup_date'] = pd.to_datetime(df['signup_date'])
```

برای بررسی نتیجه:

```
df['signup_date'].dtype
```

نوع داده پس از تبدیل:

```
datetime64
```

شد.

این تغییر باعث می‌شود تحلیل‌های زمانی مانند استخراج سال، ماه، فاصله زمانی و مرتب‌سازی بر اساس تاریخ به شکل صحیح انجام شوند.

13. تبدیل AGE به INTEGER

پس از جایگزینی مقدار Missing، ستون age دیگر نیازی به ذخیره شدن به صورت Float نداشت.

بنابراین نوع آن به Integer تبدیل شد:

```
df['age'] = df['age'].astype(int)
```

این تغییر از نظر مفهومی نیز مناسب است، زیرا سن در این دیتاست به صورت عدد صحیح ثبت شده است.

14. بررسی مقادیر دسته‌ای

برای بررسی مقادیر مختلف ستون‌های دسته‌ای از unique() و value_counts() استفاده شد.

برای مثال:

```
df['gender'].unique()
```

و:

```
df['payment_method'].value_counts()
```

بررسی مشابه برای ستون‌های:

```
categorical_columns = [
```

```
'gender',
```

```
'city',
```

```
'province',
```

```
'membership_tier',
```

```
'payment_method',
```



```
'device',  
'discount_used'  
]
```

```
for column in categorical_columns:
```

```
    print(column)
```

```
    print(df[column].unique())
```

انجام شد.

مقادیر مشاهده شده در ستون های اصلی با مقادیر مورد انتظار سازگار بودند.

در ستون `payment_method` نیز سه مقدار زیر مشاهده شد:

- Card
- Online Wallet
- Cash

و در ستون `device`:

- Android
- Web
- iPhone

در بررسی انجام شده، ناسازگاری متنی واضحی مانند فاصله اضافی یا نام گذاری متفاوت برای یک مقدار مشاهده نشد.

بنابراین نیازی به اصلاح این ستون ها وجود نداشت.

15. بررسی OUTLIER ها

پس از بررسی `Missing Values` و `Data Types`، `Outlier` های ستون های عددی بررسی شدند.

برای شناسایی `Outlier` ها از روش `IQR` استفاده شد.

فرمول: `IQR`

$IQR = Q3 - Q1$

و محدوده `Outlier` ها:

$Lower\ Bound = Q1 - 1.5 \times IQR$

$$\text{Upper Bound} = Q3 + 1.5 \times \text{IQR}$$

برای انجام این بررسی، کدی مشابه زیر استفاده شد:

$$Q1 = \text{df}[\text{column}].\text{quantile}(0.25)$$

$$Q3 = \text{df}[\text{column}].\text{quantile}(0.75)$$

$$\text{IQR} = Q3 - Q1$$

$$\text{lower_bound} = Q1 - 1.5 \times \text{IQR}$$

$$\text{upper_bound} = Q3 + 1.5 \times \text{IQR}$$

```
outliers = df[
    (df[column] < lower_bound) |
    (df[column] > upper_bound)
]
```

این بررسی برای ستون‌های عددی زیر انجام شد:

```
numeric_columns = [
    'age',
    'purchase_count',
    'avg_order_value',
    'total_spending',
    'last_purchase_days',
    'returned_items',
    'satisfaction_score'
]
```

16. نتایج OUTLIER DETECTION

نتایج بررسی به صورت زیر بود:

تعداد	Outlier	Upper Bound	Lower Bound	IQR	Q3	Q1	ستون
1		97.375	-7.625	26.25	58	31.75	age
0		45.125	-9.875	13.75	24.5	10.75	purchase_count
0		648.0625	-215.8175	215.97	324.1075	108.1375	avg_order_value
5		9979.8225	-4120.6375	3525.115	4692.15	1167.035	total_spending
0		483	-75	139.5	273.75	134.25	last_purchase_days
0		14.5	-5.5	5	7	2	returned_items
0		7	-1	2	4	2	satisfaction_score

در این بررسی یک Outlier در ستون age و پنج Outlier در ستون total_spending شناسایی شد.

نکته مهم این است که Outlier بودن یک مقدار به تنهایی به معنی اشتباه بودن آن نیست. بنابراین موارد شناسایی شده به صورت جداگانه بررسی شدند.

17. بررسی Outlier ستون AGE

رکورد مربوط به Outlier ستون age با کد زیر استخراج شد:

```
df[df['age'] > 97.375]
```

رکورد مربوط به Customer 1010 بود:

• Customer ID: 1010

• Name: Arash

• Age: 145

• City: Karaj

• Province: Alborz

• Membership Tier: Silver

مقدار 145 از نظر آماری خارج از Upper Bound برابر با 97.375 بود.

علاوه بر آن، این مقدار از نظر منطقی نیز برای سن مشتری نامعتبر در نظر گرفته شد.

بنابراین مقدار 145 اصلاح شد.

از آنجا که مقدار واقعی قابل استنتاج نبود، Median سن یعنی 45 جایگزین شد:

```
df.loc[df['customer_id'] == 1010, 'age'] = 45
```

در نتیجه:

145 → 45

18. بررسی OUTLIER های TOTAL_SPENDING

برای مشاهده Outlier های total_spending از همان روش IQR استفاده شد:

```
Q1 = df['total_spending'].quantile(0.25)
```

```
Q3 = df['total_spending'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
outliers = df[
```

```
(df['total_spending'] < lower_bound) |
```

```
(df['total_spending'] > upper_bound)
```

```
]
```

پنج رکورد به عنوان Outlier شناسایی شدند.

اما قبل از حذف آن‌ها، رابطه مالی بین ستون‌ها بررسی شد.

19. ایجاد CALCULATED_SPENDING

برای بررسی سازگاری total_spending با سایر ستون‌ها، یک ستون موقت به نام calculated_spending ایجاد شد:

```
df['calculated_spending'] = (
```

```
df['purchase_count'] *
```

```
df['avg_order_value']
```

```
)
```

سپس مقدار محاسبه‌شده با مقدار موجود مقایسه شد.

برای مشاهده نتایج:

```
df[
    [
        'customer_id',
        'purchase_count',
        'avg_order_value',
        'total_spending',
        'calculated_spending'
    ]
]
```

همچنین برای بررسی اختلاف:

```
df['spending_difference'] = (
    df['total_spending'] -
    df['calculated_spending']
)
```

این بررسی مشخص کرد که برخی Outlier های آماری از نظر منطقی کاملاً معتبر هستند.

20. بررسی Outlier های معتبر

برای Customer های 1009، 1012، 1044 و 1057 مقدار total_spending با مقدار محاسبه شده سازگار بود.

برای مثال:

Customer 1009

$$34 \times 341.63 = 11615.42$$

و مقدار ثبت شده نیز:

11615.42

بود.

بنابراین این رکوردها با وجود Outlier بودن از نظر آماری، خطای داده‌ای محسوب نشدند و حذف یا اصلاح نشدند.

این موضوع نشان می‌دهد که در Data Cleaning نباید Outlier ها را بدون بررسی منطقی حذف کرد.

21. اصلاح TOTAL_SPENDING برای CUSTOMER 1030

در بررسی Customer 1030 مشخص شد:

`purchase_count = 26`

`avg_order_value = 156.89`

`total_spending = 25000`

اما مقدار واقعی بر اساس رابطه بین ستون‌ها باید برابر باشد با:

$$26 \times 156.89 = 4079.14$$

بنابراین مقدار 25000 اشتباه تشخیص داده شد.

برای اصلاح:

```
df.loc[df['customer_id'] == 1030, 'total_spending'] = (  
    df.loc[df['customer_id'] == 1030, 'purchase_count']  
    * df.loc[df['customer_id'] == 1030, 'avg_order_value']  
)
```

در نتیجه:

25000 → 4079.14

22. بررسی سازگاری PURCHASE_COUNT و RETURNED_ITEMS

یکی دیگر از کنترل‌های مهم، بررسی رابطه بین تعداد خرید و تعداد اقالام برگشتی بود.

Rule منطقی تعریف شده:

`returned_items ≤ purchase_count`

برای شناسایی رکوردهایی که این شرط را نقض می‌کنند:

```
df[  
    df['returned_items'] >  
    df['purchase_count']  
]
```

شش رکورد ناسازگار شناسایی شد.

رکوردهای ناسازگار عبارت بودند از:

Customer ID	Name	Purchase Count	Returned Items
1008	Reza	3	8
1015	Kimia	3	8
1024	Sina	0	2
1029	Ali	2	4
1037	Zahra	1	7
1056	Kimia	1	4

از آنجا که مقدار واقعی قابل استنتاج نبود، تصمیم گرفته شد Rule منطقی رعایت شود.

برای این کار می‌توان از کد زیر استفاده کرد:

```
df['returned_items'] = np.minimum(
    df['returned_items'],
    df['purchase_count']
)
```

در نتیجه:

- Customer 1008: 8 → 3
- Customer 1015: 8 → 3
- Customer 1024: 2 → 0
- Customer 1029: 4 → 2
- Customer 1037: 7 → 1
- Customer 1056: 4 → 1

این اصلاح به عنوان Rule-Based Correction در نظر گرفته شد.

برای بررسی بهتر رابطه بین خرید و برگشت کالا، نسبت اقلام برگشتی به خرید نیز محاسبه شد:

```
df['return_ratio'] = (
    df['returned_items'] /
    df['purchase_count']
)
```

در این مرحله مشخص شد که در برخی رکوردها نسبت برگشت بسیار بالا است.

البته برای مشتریانی که `purchase_count = 0` دارند، تقسیم مستقیم می‌تواند باعث ایجاد مقدار نامعتبر شود. بنابراین این نسبت باید در تحلیل‌های بعدی با توجه به مقدار صفر در مخرج مدیریت شود.

این بررسی به شناسایی ناسازگاری‌های منطقی در داده کمک کرد، اما مبنای اصلی اصلاح `returned_items` همان Rule تعریف‌شده بین تعداد خرید و تعداد اقلام برگشتی بود.

25. کنترل نهایی سازگاری مالی

پس از اصلاح داده‌ها، رابطه بین:

- `purchase_count`
- `avg_order_value`
- `total_spending`

دوباره بررسی شد.

برای این کار:

```
df['calculated_spending'] = (
    df['purchase_count'] *
    df['avg_order_value']
)
```

و سپس اختلاف محاسبه شد:

```
df['spending_difference'] = (
    df['total_spending'] -
    df['calculated_spending']
)
```

برای بررسی اختلاف‌های واقعی، از مقایسه عددی با یک `Tolerance` کوچک استفاده شد:


```

df[
    ~np.isclose(
        df['total_spending'],
        df['calculated_spending'],
        atol=0.01
    )
]

```

این روش برای جلوگیری از مشکلات ناشی از محاسبات اعشاری استفاده شد.

نتیجه نشان داد که پس از اصلاح Customer 1030 و تکمیل Customer 1040 ، مقدار total_spending با رابطه محاسباتی مورد انتظار سازگار است.

FINAL DATA QUALITY CHECK .26

پس از تمام اصلاحات، یک بررسی نهایی روی دیتاست انجام شد.

برای: Missing Values

```
df.isnull().sum()
```

برای: Duplicate

```
df.duplicated().sum()
```

برای بررسی سن‌های غیرمنطقی:

```

df[
    (df['age'] < 18) |
    (df['age'] > 100)
]

```

برای بررسی تعداد اقلام برگشتی بیشتر از تعداد خرید:

```

df[
    df['returned_items'] >
    df['purchase_count']
]

```

برای بررسی امتیاز رضایت:

```

df[
(df['satisfaction_score'] < 1) |
(df['satisfaction_score'] > 5)
]
و برای بررسی سازگاری مالی:
np.isclose(
df['total_spending'],
df['purchase_count'] * df['avg_order_value'],
atol=0.01
)

```

این کنترل‌ها نشان دادند که مشکلات اصلی شناسایی‌شده در مراحل قبل پس از پاک‌سازی برطرف شده‌اند.

27. خلاصه تغییرات انجام‌شده

مشکل	تعداد	اقدام انجام‌شده
Duplicate	1	حذف رکورد تکراری
Missing age	1	جایگزینی با Median = 45
Missing total_spending	1	محاسبه مقدار 1280.44
signup_date به صورت String	کل ستون	تبدیل به Datetime
age به صورت Float	کل ستون	تبدیل به Integer
Age Outlier	1	اصلاح 145 به 45
total_spending Outlier	5	بررسی موردی
total_spending اشتباه	1	اصلاح 25000 به 4079.14
returned_items > purchase_count	6	اصلاح بر اساس Rule منطقی

28. ذخیره دیتاست پاک‌سازی‌شده

پس از پایان عملیات Data Cleaning، دیتاست نهایی در یک فایل Excel ذخیره شد.

کد مورد استفاده:

```
df.to_excel(
    'cleaned_dataset.xlsx',
    index=False
)
```

پارامتر `index=False` باعث می‌شود Index داخلی Pandas به عنوان یک ستون اضافی وارد فایل Excel نشود.

فایل خروجی:

cleaned_dataset.xlsx

است.

29. وضعیت دیتاست قبل و بعد از پاک‌سازی

در ابتدای پروژه دیتاست دارای مشکلات مختلفی در زمینه کیفیت داده بود که در طول مراحل بررسی شناسایی شدند.

مهم‌ترین مشکلات عبارت بودند از:

- وجود یک رکورد تکراری
- وجود دو مقدار Missing
- ذخیره شدن تاریخ‌ها به صورت String
- وجود سن غیرمنطقی 145
- وجود Outlier در total_spending
- وجود یک مقدار ناسازگار در total_spending
- وجود شش مورد ناسازگاری بین returned_items و purchase_count

پس از انجام پاک‌سازی:

- Duplicate‌ها حذف شدند.
- Missing Value‌ها برطرف شدند.
- نوع داده تاریخ اصلاح شد.

- سن غیرمنطقی اصلاح شد.
- مقدار اشتباه total_spending اصلاح شد.
- Outlier های معتبر بدون حذف حفظ شدند.
- ناسازگاری بین تعداد خرید و اقلام برگشتی اصلاح شد.
- کنترل های نهایی Data Quality انجام شد.

30. محدودیت های پروژه

در برخی موارد مقدار واقعی داده از اطلاعات موجود قابل تشخیص نبود.

برای مثال در مورد Missing Value مربوط به age برای 1020 Customer، اطلاعات دیگری برای محاسبه سن وجود نداشت. بنابراین Median به عنوان روش جایگزینی استفاده شد.

همچنین در مورد returned_items برای شش رکورد ناسازگار، مقدار واقعی قابل تعیین نبود. اصلاح این موارد بر اساس یک Rule منطقی انجام شد و نه بر اساس اطلاعات قطعی از منبع اصلی داده.

بنابراین این موارد باید در مستندات پروژه به عنوان تصمیم های مبتنی بر منطق داده در نظر گرفته شوند.

همچنین Outlier Detection بر اساس روش IQR انجام شد و تصمیم گیری نهایی درباره Outlier ها با بررسی مقدار واقعی و رابطه آن با سایر ستون ها صورت گرفت.

31. پیشنهاد برای مراحل بعدی

پس از آماده شدن دیتاست پاک سازی شده، می توان از آن برای مراحل بعدی تحلیل استفاده کرد.

برخی تحلیل های پیشنهادی عبارتند از:

- بررسی رفتار خرید مشتریان
- بررسی ارتباط Membership Tier با میزان خرید
- بررسی ارتباط Satisfaction Score با رفتار خرید
- بررسی اثر استفاده از Discount بر خرید
- مقایسه روش های پرداخت
- بررسی رفتار مشتریان بر اساس شهر و استان
- بررسی مشتریان با بیشترین Total Spending
- بررسی مشتریانی که مدت زیادی از آخرین خریدشان گذشته است

- بررسی ارتباط تعداد خرید و میزان هزینه کرد
- طراحی داشبورد مدیریتی
- Customer Segmentation
- RFM Analysis

32. نتیجه‌گیری

در این پروژه یک دیتاست خام مشتریان از نظر کیفیت داده بررسی و پاک‌سازی شد.

در مرحله اول ساختار داده و انواع متغیرها بررسی شدند. سپس Duplicate Records ، Missing Values ، Data Types ، مقادیر دسته‌ای، Outlier ها و سازگاری بین ستون‌ها مورد بررسی قرار گرفتند.

در مجموع، یک رکورد تکراری حذف شد، دو Missing Value برطرف شدند، نوع داده تاریخ اصلاح شد، یک سن غیرمنطقی اصلاح شد، یک مقدار اشتباه در total_spending اصلاح شد و شش مورد ناسازگاری بین تعداد خرید و اقلام برگشتی مدیریت شدند.

در مورد Outlier ها نیز تصمیم به حذف خودکار گرفته نشد و هر مورد به صورت جداگانه بررسی شد. چهار Outlier معتبر در total_spending حفظ شدند، در حالی که مقدار ناسازگار Customer 1030 اصلاح شد.

در این پروژه برای هر مرحله علاوه بر بررسی نتایج، از کدهای Python و Pandas برای شناسایی و اصلاح مشکلات استفاده شد. این موضوع باعث شد فرایند Data Cleaning قابل تکرار و قابل بررسی باشد.

مهم‌ترین نتیجه این پروژه برای من این بود که Data Cleaning فقط به معنی حذف داده‌های اشتباه نیست. بخش مهم‌تر این فرایند، تشخیص این موضوع است که کدام داده واقعاً اشتباه است، کدام داده فقط غیرعادی است و برای هر تغییر چه دلیل منطقی می‌توان ارائه کرد.

دیتاست پاک‌سازی شده اکنون می‌تواند به عنوان ورودی مراحل بعدی مانند EDA ، Visualization ، تحلیل رفتار مشتری، تحلیل فروش و طراحی داشبورد مورد استفاده قرار گیرد.

33. فایل‌های خروجی پروژه

خروجی نهایی پروژه شامل فایل‌های زیر است:

1. cleaned_dataset.xlsx

نسخه پاک‌سازی شده دیتاست که پس از انجام تمام اصلاحات ذخیره شده است.

2. data_cleaning.ipynb

فایل Jupyter Notebook که شامل کدهای مربوط به:

- بررسی اولیه داده
- بررسی Duplicate
- بررسی Missing Values

- بررسی Data Types
- تبدیل تاریخ
- بررسی مقادیر دسته‌ای
- شناسایی Outlier
- بررسی سازگاری بین ستون‌ها
- اصلاح داده‌ها
- اعتبارسنجی نهایی

است.

3. README.md

فایل مستندات پروژه که در آن مشکلات شناسایی شده، تصمیم‌های مربوط به پاک‌سازی و تغییرات اعمال شده ثبت شده‌اند.

4. analysis_report.pdf

گزارش نهایی پروژه که فرایند Data Cleaning، نتایج بررسی‌ها و تصمیم‌های گرفته‌شده را توضیح می‌دهد.

34. ساختار نهایی پروژه

ساختار پیشنهادی فایل‌های پروژه به صورت زیر است:

```
project-01-data-cleaning/
|
└─ data/
|   └─ cleaned_dataset.xlsx
|
|   └─ notebook/
|       └─ data_cleaning.ipynb
|
|   └─ report/
|       └─ analysis_report.pdf
|
└─ README.md
```

این ساختار باعث می‌شود داده خروجی، کدهای پروژه، گزارش و مستندات از یکدیگر جدا باشند و پروژه برای بررسی و استفاده مجدد مرتب و قابل فهم باقی بماند.